Nakijk 02_02 — V2

NAKIJK V2 · Hoofdstuk 02 · Volledige CRUD · bij Oefening_02_02.md · 14 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Models/Ventilator.cs` (11 oktober) en `Controllers/VentilatorController.cs` (12 oktober, 17:52).

> **OPMERKING**
>
> Je vraag: je POST werkt niet en je begrijpt niet waarom. Je krijgt 201 Created, maar bij een volgend verzoek is de nieuwe ventilator weg. Dit verslag gaat vooral daarover.
>
> Je controller is niet meer gewijzigd sinds de V1-nakijk. Het probleem stond daar onder A1. Hier leg ik het anders uit, stap voor stap, met een manier om het zelf te zien in Visual Studio.
>
> Nagekeken is wat je al gemaakt hebt: het model, de twee GET's en de POST. De controles van punt 4 en de punten 5 tot 8 staan onderaan als nog te doen, niet als fout.
>
> Ik kon je API niet starten, want op deze werkruimte staat geen .NET. Ik heb je code gelezen en elk verzoek op papier nagelopen.

In één oogopslag

Je POST zelf is goed. Hij neemt de ventilator uit de body, geeft hem een nieuw Id, voegt hem toe en geeft 201 met het juiste adres. Het probleem zit niet in de POST, maar in de lijst waar hij de ventilator in zet: **die lijst bestaat maar zolang één verzoek duurt** (A1). Bij het volgende verzoek staat er een nieuwe lijst, met alleen de vier vaste ventilatoren.

Punt	Wat de opgave vraagt	Status
1	het model `Ventilator`	✓ in orde
2	de controller op `api/ventilator`, met ventilatoren die bewaard blijven	✗ zie A1
3	`GET api/ventilator` en `GET api/ventilator/{id}`	½ zie A2
4	`POST api/ventilator` met 201 en het adres	½ het toevoegen zelf is goed, maar zie A1
4	de drie controles met 400	nog te doen
5 tot 8	bijwerken, verwijderen, testen en vervangen	nog te doen

A1. Waarom je nieuwe ventilator weer weg is

Waar: `VentilatorController.cs`, de declaratie van de lijst op regel 10.

Wat je ziet

Verzoek	Je API antwoordt
`POST api/ventilator` met een geldige ventilator	201 Created, Id 5, adres *api/Ventilator/5*
`GET api/ventilator`	de vier oorspronkelijke ventilatoren, de nieuwe staat er niet bij
`GET api/ventilator/5`	404 met *We vonden geen ventilator met Id 5*
nog eens `POST api/ventilator`	201 Created, **opnieuw Id 5**

Die laatste regel is het beste bewijs. Was de eerste nieuwe ventilator bewaard gebleven, dan was het hoogste Id 5 en kreeg de tweede Id 6.

Wat er gebeurt, verzoek per verzoek

Je controller is een klasse. Voor **elk verzoek** maakt ASP.NET een nieuw object van die klasse, laat het verzoek afhandelen en gooit het object daarna weg.

Bij het maken van een object voert C# eerst alle velden uit die een beginwaarde hebben. Regel 10 is zo'n veld: daar maak je een nieuwe lijst met de vier ventilatoren. **Regel 10 wordt dus bij elk verzoek opnieuw uitgevoerd**, en elk verzoek krijgt een eigen, verse lijst.

Verzoek	Wat er gebeurt	In de lijst	Na het antwoord
1. POST	object 1 wordt gemaakt, regel 10 maakt lijst 1 met vier ventilatoren, je POST voegt Id 5 toe	5 ventilatoren	object 1 en lijst 1 worden weggegooid
2. GET alles	object 2 wordt gemaakt, regel 10 maakt lijst 2	4 ventilatoren	weggegooid
3. GET 5	object 3, met lijst 3	4 ventilatoren, dus geen Id 5: 404	weggegooid
4. POST	object 4, met lijst 4; het hoogste Id is weer 4	5 ventilatoren, de nieuwe met Id 5	weggegooid

Je POST heeft dus telkens gewerkt, maar in een lijst die meteen daarna verdween.

Je kan het vergelijken met een blad waarop vier ventilatoren voorgedrukt staan. Bij elk verzoek krijgt je controller een nieuw blad. Je POST schrijft er netjes een vijfde bij, maar na het antwoord gaat het blad in de papiermand. Het volgende verzoek krijgt weer een nieuw voorgedrukt blad.

Waarom ASP.NET dat zo doet

Een nieuwe controller per verzoek is een bewuste keuze. Gebruiken twee mensen je API tegelijk, dan krijgt elk zijn eigen object en zitten ze elkaar niet in de weg. Er blijft ook niets van een vorig verzoek per ongeluk in je controller hangen.

Het gevolg: alles wat langer moet bestaan dan één verzoek, mag niet bij het **object** van je controller horen.

Het begrip: hoe lang bestaat een variabele?

Waar ze staat	Hoe lang ze bestaat	In je controller
in een methode	één aanroep van die methode	`ventilator` in `GetVentilatorById`
als veld van het object	zo lang als het object, en bij een controller is dat één verzoek	je lijst, zoals ze nu staat
als veld van de klasse	één keer voor de hele klasse, zo lang als de applicatie draait	wat je lijst moet worden

Een veld van het object bestaat **per object**: tien verzoeken, tien lijsten. Een veld van de klasse bestaat **één keer**, en alle objecten gebruiken hetzelfde exemplaar.

Zelf zien in Visual Studio

Je kan dit met breakpoints zelf nagaan. Een breakpoint zet je door in de grijze rand links van een regel te klikken.

1. Zet een breakpoint op regel 10 en start je API met F5. Doe in Scalar een GET op `api/ventilator`. Visual Studio stopt op regel 10. Druk op F5 om verder te gaan. Doe dezelfde GET nog eens: hij stopt opnieuw op regel 10. **Elke stop is een nieuwe lijst.**
2. Zet een breakpoint op regel 45, de return van je POST. Doe een POST. Hou je muis boven `Ventilatoren` als Visual Studio stopt: er staan er 5 in.
3. Zet een breakpoint op regel 21, in `GetVentilatoren`. Doe een GET. Hou je muis opnieuw boven `Ventilatoren`: er staan er 4 in. Dezelfde naam, een andere lijst.

Richting

- Vergelijk de declaratie van de lijst in de eerste versie van de `ProductController` in les 02 met de volledige `ProductController` achteraan dezelfde les. Er staat één woord meer.
- Je leerboek, hoofdstuk 24, legt uit wat dat woord doet.
- Dat woord hoort bij de lijst. De methodes en de klasse zelf blijven zoals ze zijn.

Wat niet helpt

- **De lijst in je methode zetten.** Dan wordt ze een variabele van de methode, en leeft ze nog korter: één aanroep. Je andere methodes zien ze dan ook niet meer.
- **Dat woord voor de hele controller zetten.** Een klasse met dat woord kan niet van `ControllerBase` erven, en ASP.NET kan er geen object van maken. Dat compileert niet: melding **CS0713**.

Hoe je weet dat het opgelost is

- Het breakpoint op regel 10 stopt nog maar **één keer** zolang je API draait, bij het eerste verzoek dat de lijst nodig heeft. Daarna niet meer.
- POST, en daarna `GET api/ventilator`: vijf ventilatoren.
- `GET api/ventilator/5`: 200 met je nieuwe ventilator.
- Een tweede POST krijgt Id 6.

Herstart je de API, dan staan er weer vier. Dat is normaal: de lijst staat in het geheugen zolang de applicatie draait. Gegevens die een herstart overleven, komen pas met de database in les 04.

A2. Het bericht bij een onbekend Id mist nog een punt

Waar: `GetVentilatorById`, regel 31. Dit stond ook in V1 en is nog open.

De opgave vraagt *We vonden geen ventilator met Id {id}.*, met een punt achteraan. In jouw bericht ontbreekt het. Een mens leest erover, maar het nakijkscript vergelijkt het bericht letterlijk en ziet twee verschillende teksten.

Richting. Leg je bericht teken voor teken naast dat van de opgave.

Nog te doen

Dit heb je nog niet gemaakt, en dat is dus geen fout. De richting per punt staat in V1, bij A3 tot A6.

Punt	Wat er nog moet komen	In V1
4	de drie controles met 400: leeg merk, afmeting niet 120 of 140, prijs 0 of lager	A3
5	`PUT api/ventilator/{id}` met 204, 404 en 400	A4
6	`DELETE api/ventilator/{id}` met 204 en 404	A5
7	de twee testen: 92 mm geeft 400, een tweede DELETE geeft 404	A3 en A5
8	`PUT api/ventilator/vervang/{id}`	A6

> **BELANGRIJK**
>
> Los A1 eerst op. Zonder bewaarde lijst kan je punt 5 tot 8 niet testen. Een verwijderde ventilator staat er bij het volgende verzoek gewoon weer, en een DELETE geeft dan twee keer 204 in plaats van 204 en daarna 404.

Opkuis, zonder invloed op het antwoord

Ook dit stond in V1 en is nog open.

Waar	Wat	Richting
regel 10	Het veld heet `Ventilatoren`, met een hoofdletter. Velden beginnen in C# met een kleine letter; de les schrijft `laptops`.	Pas je A1 aan, neem dit dan in één keer mee.
regel 1 en 2	De twee `using`-regels staan ook al in `GlobalUsings.cs`. Visual Studio toont ze daarom grijs.	Weghalen; ze doen niets.
regel 27 en 40	De parameter in je lambda heet `fan`, terwijl de rest van je code Nederlands is.	Kies één taal. De les gebruikt een korte letter, zoals `x`.

Wat goed is, en waarom

- **De data komt uit de body.** Je parameter is een `Ventilator`, een eigen klasse. `[ApiController]` haalt zo'n parameter vanzelf uit de body van het verzoek, en dat is wat punt 4 vraagt.
- **Het Id bepaal je zelf**, met het hoogste Id plus één, en een meegestuurd Id overschrijf je. Dat vraagt de opgave letterlijk. Zodra A1 opgelost is, werkt die telling ook over meerdere verzoeken heen.
- `CreatedAtAction` **met** `nameof`**.** Precies zoals de les. Je antwoord is 201, de nieuwe ventilator staat in de body en het adres in de Location-header. `nameof` laat je compiler een fout geven als de methode ooit een andere naam krijgt, in plaats van een link die stilletjes niet meer werkt.
- `Ventilator?` **op regel 27.** Het vraagteken klopt, want `FirstOrDefault` kan niets vinden. Zo vermijd je de waarschuwing CS8600 die je bij 02_01 had.
- **De returntypes** kloppen met wat je teruggeeft: een lijst bij het ophalen van alles, één ventilator bij de andere twee.
- **Je testdata** heeft beide toegelaten afmetingen en verschillende verlichting. Daar heb je straks iets aan bij het testen van punt 5 tot 8.

Voor zover ik kan nagaan door te lezen, geeft je code geen enkele compilerwaarschuwing.

Zelf nakijken

Doe na het oplossen van A1 en A2 deze verzoeken, in deze volgorde. Een antwoord klopt pas als statuscode én bericht letterlijk overeenkomen met de opgave.

Verzoek	Wat je moet zien	Hoort bij
`POST api/ventilator` met een geldige ventilator	201, Id 5, adres *api/Ventilator/5*	punt 4
`GET api/ventilator`	vijf ventilatoren	A1
`GET api/ventilator/5`	200 met je nieuwe ventilator	A1
nog een geldige `POST`	201, Id 6	A1
`GET api/ventilator/99`	404 met *We vonden geen ventilator met Id 99.*	A2

Met het nakijkscript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_02 -Project
"...\ProgrammingAdvancedOefeningen\ProgrammingAdvancedOefeningen"
```

Het script stuurt alle verzoeken uit de opgave, dus ook die van punt 5 tot 8. Die staan zolang als fout in het verslag. Kijk voorlopig alleen naar punt 1 tot de eerste POST van punt 4.

Vragen die je je misschien stelt

Waarom zag ik dit niet bij 02_01? Daar las je alleen. Elk verzoek kreeg een verse lijst, maar die was altijd dezelfde, dus je merkte niets. Zodra je iets toevoegt, wijzigt of verwijdert, moet de lijst langer bestaan dan één verzoek.

Het adres in de Location-header klopt toch? Waarom geeft het dan een 404? Het adres wordt gemaakt uit je route en het nieuwe Id. Het controleert niet of de ventilator er bij een volgend verzoek nog is. Het adres is juist, alleen staat er niets meer achter.

Is één lijst voor alle verzoeken samen veilig als twee mensen tegelijk iets toevoegen? Niet helemaal. Een `List` is niet gemaakt om door meerdere verzoeken tegelijk gewijzigd te worden. Voor deze oefening is dat geen probleem, want jij test alleen. Vanaf les 04 komt de database, en die regelt dat voor je.

Waarom maakt de les dan niet meteen een database? Les 02 leert je de HTTP-kant: verbs, statuscodes en de body. De lijst in het geheugen is een tijdelijke vervanger, zodat je niet twee nieuwe dingen tegelijk moet leren. In les 03 en 04 wordt ze stap voor stap vervangen.